

Tuto création d'une API avec node.JS

1. Initialiser le projet

Étape N°1 : Crée un dossier de projet et entre dedans :

```
mkdir api-node-ts && cd api-node-ts
```

Étape N°2 : Initialise un projet Node.js :

```
npm init -y
```

2. Installer les dépendances

◆ Dépendances principales

Étape N°3 : Entrez cette commande

```
npm install express dotenv
```

◆ Dépendances de développement

```
npm install -D typescript ts-node @types/node @types/express nodemon
```

Explication :

- `express` → Framework web pour créer l'API.
- `dotenv` → Gestion des variables d'environnement.
- `typescript` → Ajoute le support TypeScript.
- `ts-node` → Exécute TypeScript sans compilation explicite .
- `@types/node`, `@types/express` → Définitions de types.
- `nodemon` → Recharge le serveur en cas de modification.

3. Configurer TypeScript

Étape N°4 : Entrez cette commande qui génère le fichier `tsconfig.json` :

```
npx tsc -init
```

Étape N°5 : Modifiez `tsconfig.json` :

```
{
  "compilerOptions": {
    "target": "ES6",
    "module": "CommonJS",
    "outDir": "./dist",
    "rootDir": "./src",
    "esModuleInterop": true,
```

```
"resolveJsonModule": true,
"strict": true
}
```

Explication des options de tsconfig.json

◆ 1. target: "ES6"

👉 Définit la version JavaScript à générer après la compilation.

Valeur	Description
"ES5"	Compatible avec les anciens navigateurs, mais pas de let, const et async/await.
"ES6"	Permet let, const, arrow functions, async/await.
"ESNext"	Active les dernières fonctionnalités de JavaScript.

✅ Ici, "ES6" signifie que TypeScript va compiler ton code en JavaScript ES6 (ECMAScript 2015).

◆ 2. module: "CommonJS"

👉 Définit le format des imports/exports.

Valeur	Utilisation
"CommonJS"	Utilisé par Node.js (nécessaire pour require et module.exports).
"ESNext" OU "ES6"	Utilisé pour les importations modernes (import/export).

✅ Comme on utilise Node.js, on garde "CommonJS".

◆ 3. outDir: "./dist"

👉 Définit où seront enregistrés les fichiers compilés en JavaScript.

- Fichier TypeScript (src/index.ts) → sera compilé en dist/index.js.

✅ Tous les fichiers .js compilés iront dans dist/.

◆ 4. rootDir: "./src"

👉 Spécifie où se trouvent les fichiers source TypeScript.

✅ TypeScript va compiler uniquement les fichiers de src/.

◆ 5. strict: true

👉 Active toutes les vérifications strictes de TypeScript.

Cela inclut :

- strictNullChecks : Empêche d'utiliser null ou undefined sans vérification.
- noImplicitAny : Empêche l'utilisation de variables sans type.
- strictFunctionTypes : Vérifie plus strictement les types des fonctions.

✅ C'est une bonne pratique pour éviter les erreurs !

◆ 6. esModuleInterop: true

👉 Permet d'importer des modules **CommonJS** (`require()`) comme si c'était des modules **ES6** (`import`).

Sans esModuleInterop: true (Erreur)

```
import express from 'express'; // ❌ Erreur
```

Avec esModuleInterop: true (Corrigé)

```
import express from 'express'; // ✅ Fonctionne
```

Avec esModuleInterop: true (Corrigé)

✅ Obligatoire pour éviter des erreurs avec **Express** et d'autres bibliothèques CommonJS.

◆ 7. resolveJsonModule: true

👉 Permet d'importer directement des fichiers **JSON** dans TypeScript.

Exemple :

```
import config from './config.json';
console.log(config.database.host);
```

✅ Utile si ton projet charge des **fichiers de configuration JSON**.

✅ Résumé des options

Option	Effet
"target": "ES6"	Compile TypeScript en JavaScript moderne
"module": "CommonJS"	Utilise les modules Node.js
"outDir": "./dist"	Stocke les fichiers compilés dans <code>dist/</code>
"rootDir": "./src"	Source TypeScript dans <code>src/</code>
"strict": true	Active les vérifications strictes
"esModuleInterop": true	Permet <code>import express from 'express'</code>
"resolveJsonModule": true	Permet d'importer des fichiers JSON

🔧 4. Créer l'API

Crée un dossier `src/` :

```
mkdir src
```

📌 4.1. Créer `src/index.ts`

Étape N°6 : Copiez ce fichier est le point d'entrée de l'API :

```
// Importation des modules nécessaires
import express, { Request, Response } from 'express'; // Framework Express et types pour les requêtes/réponses
import * as dotenv from 'dotenv'; // Permet de charger les variables d'environnement
import userRoutes from './routes/user.routes'; // Importe les routes utilisateurs
```

```
// Charge les variables d'environnement depuis le fichier .env
dotenv.config();

// Création de l'application Express
const app = express();

// Définition du port du serveur (utilise celui de l'environnement ou 3000 par défaut)
const PORT = process.env.PORT || 3000;

// Middleware pour parser le JSON dans les requêtes entrantes
app.use(express.json());

// Route de test pour vérifier si le serveur fonctionne
app.get('/', (req: Request, res: Response) => {
  res.send('🚀 API Node.js avec TypeScript fonctionne !'); // Réponse envoyée au client
});

// Utilisation des routes utilisateurs définies dans "user.routes.ts"
app.use('/users', userRoutes);

// Démarrage du serveur sur le port défini
app.listen(PORT, () => {
  console.log('✅ Serveur démarré sur http://localhost:${PORT}'); // Message de confirmation
  dans la console
});
```

Explications

```
import express, { Request, Response } from 'express'; // Framework Express et types pour les
requêtes/réponses
```

👉 Importe le module express

👉 Request et Response sont des types fournis par Express pour mieux gérer les requêtes et réponses en TypeScript.

Par exemple :

```
app.get('/', (req: Request, res: Response) => {
  res.send('🚀 API Node.js avec TypeScript fonctionne !'); // Réponse envoyée au client
});
```

Request Type qui représente la requête HTTP (ex: req.body)

Response Type qui représente la réponse HTTP (ex: res.send())

```
import * as dotenv from 'dotenv'; // Permet de charger les variables d'environnement
```

- **dotenv** → Permet de charger les **variables d'environnement** depuis un fichier .env.

```
import userRoutes from './routes/user.routes'; // Importe les routes utilisateurs
```

userRoutes → Importe un fichier qui gère les routes utilisateurs.

Dans Express, il est **bonne pratique** de séparer les routes en différents fichiers pour **garder un code organisé**.

👉 **Au lieu de définir toutes les routes dans `index.ts`**, on les met dans un fichier dédié (`user.routes.ts`) et on l'importe.

```
// Charge les variables d'environnement depuis le fichier .env
dotenv.config();
```

```
// Création de l'application Express
const app = express();
```

Créer un serveur basé sur express

```
// Définition du port du serveur (utilise celui de l'environnement ou 3000 par défaut)
const PORT = process.env.PORT || 3000;
```

```
// Middleware pour parser le JSON dans les requêtes entrantes
app.use(express.json());
```

ajoute un middleware qui permet à Express de **parser** (analyser) le corps (`body`) des requêtes **JSON**.

Sans cette ligne, les données envoyées en JSON via une requête **POST** ou **PUT** ne seraient pas accessibles via `req.body`.

```
// Route de test pour vérifier si le serveur fonctionne
app.get('/', (req: Request, res: Response) => {
  res.send('🚀 API Node.js avec TypeScript fonctionne !'); // Réponse envoyée au client
});
```

- ◆ **app.get()** → Définit une **route GET** dans Express.
- ◆ **'/'** → Correspond à la route **racine** (`http://localhost:3000/`).
- ◆ **Callback (req, res) => {...}** → Fonction qui **gère la requête et envoie une réponse**.
- ◆ **Paramètres**.
 - **req: Request** → Objet représentant la **requête HTTP** envoyée par le client.
 - Contient des infos comme les **paramètres**, **headers**, **body**, etc.
 - **res: Response** → Objet représentant la **réponse HTTP** envoyée par le serveur.
 - Contient des méthodes comme `.send()`, `.json()`, `.status()`.

```
// Utilisation des routes utilisateurs définies dans "user.routes.ts"
app.use('/users', userRoutes);
```

permet de **monter** (`use()`) un ensemble de **routes dédiées aux utilisateurs** sous le chemin `/users`.

Cela signifie que toutes les requêtes commençant par `/users` seront **redirigées** vers le fichier `user.routes.ts`, qui contient la définition des routes liées aux utilisateurs.

```
// Démarrage du serveur sur le port défini
app.listen(PORT, () => {
  console.log('✅ Serveur démarré sur http://localhost:${PORT}'); // Message de confirmation dans la console
});
```

5. Ajouter un routeur

Étape N°7 : Crée un dossier `src/routes/` et un fichier `user.routes.ts` :

Étape N°8 : **Écrivez :**  `src/routes/user.routes.ts`

```
// Importation du module Router d'Express pour gérer les routes
import { Router } from 'express';

// Importation des contrôleurs qui gèrent la logique métier pour les utilisateurs
import { getUsers, addUser } from '../controllers/user.controller';

// Création d'un routeur Express
const router = Router();

/**
 * Route GET /users
 * Description : Récupère la liste des utilisateurs
 * Contrôleur associé : getUsers (défini dans user.controller.ts)
 */
router.get('/', getUsers);

/**
 * Route POST /users
 * Description : Ajoute un nouvel utilisateur
 * Contrôleur associé : addUser (défini dans user.controller.ts)
 */
router.post('/', addUser);

// Exportation du routeur pour l'utiliser dans index.ts
export default router;
```

Explications

Étape N°9 : Expliquez chaque ligne comme précédemment

```
// Importation des contrôleurs qui gèrent la logique métier pour les utilisateurs
import { getUsers, addUser } from '../controllers/user.controller';
```

```
// Création d'un routeur Express
const router = Router();
```

```
/**
 * Route GET /users
 * Description : Récupère la liste des utilisateurs
 * Contrôleur associé : getUsers (défini dans user.controller.ts)
 */
router.get('/', getUsers);
```

```
/**
 * Route POST /users
 * Description : Ajoute un nouvel utilisateur
 * Contrôleur associé : addUser (défini dans user.controller.ts)
 */
router.post('/', addUser);
```

```
// Exportation du routeur pour l'utiliser dans index.ts
export default router;
```

6. Ajouter un contrôleur

Étape N°10: Créez un dossier `src/controllers/` et un fichier `user.controller.ts` :

 `src/controllers/user.controller.ts`

```
// Importation des types Request et Response depuis Express
// Request : représente la requête HTTP reçue
// Response : représente la réponse HTTP envoyée au client
import { Request, Response } from 'express';

/**
 * Contrôleur pour la route GET /users
 * Description : Renvoie un message avec la liste des utilisateurs (simulation)
 * @param req - Objet représentant la requête HTTP (non utilisé ici)
 * @param res - Objet permettant d'envoyer une réponse HTTP
 */
export const getUsers = (req: Request, res: Response) => {
  res.json({ message: 'Liste des utilisateurs' }); // Envoie une réponse JSON avec un message
};

/**
 * Contrôleur pour la route POST /users
 * Description : Ajoute un nouvel utilisateur en récupérant les données du corps de la requête
 * @param req - Objet représentant la requête HTTP contenant les données utilisateur
 * @param res - Objet permettant d'envoyer une réponse HTTP
 */
export const addUser = (req: Request, res: Response) => {
```

```
const { name, email } = req.body; // Récupération des données envoyées dans le corps de la requête

// Envoie une réponse JSON confirmant l'ajout de l'utilisateur
res.json({ message: `Utilisateur ${name} ajouté avec succès !`, email });
};
```

Explications

Étape N°11: Expliquez chaque ligne comme précédemment

```
// Importation des types Request et Response depuis Express
// Request : représente la requête HTTP reçue
// Response : représente la réponse HTTP envoyée au client
import { Request, Response } from 'express';
```

```
/**
 * Contrôleur pour la route GET /users
 * Description : Renvoie un message avec la liste des utilisateurs (simulation)
 * @param req - Objet représentant la requête HTTP (non utilisé ici)
 * @param res - Objet permettant d'envoyer une réponse HTTP
 */
export const getUsers = (req: Request, res: Response) => {
  res.json({ message: 'Liste des utilisateurs' }); // Envoie une réponse JSON avec un message
};
```

```
/**
 * Contrôleur pour la route POST /users
 * Description : Ajoute un nouvel utilisateur en récupérant les données du corps de la requête
 * @param req - Objet représentant la requête HTTP contenant les données utilisateur
 * @param res - Objet permettant d'envoyer une réponse HTTP
 */
export const addUser = (req: Request, res: Response) => {
  const { name, email } = req.body; // Récupération des données envoyées dans le corps de la requête
```

```
// Envoie une réponse JSON confirmant l'ajout de l'utilisateur
res.json({ message: `Utilisateur ${name} ajouté avec succès !`, email });
};
```

7. Ajouter un fichier .env

Étape N°12: Crée un fichier .env à la racine du projet :

```
 .env
PORT=4000
```


⚡ 8. Configurer Nodemon

Étape N°13 : Pour éviter de redémarrer le serveur à chaque modification, ajoute un fichier `nodemon.json` :

📌 `nodemon.json`

```
{
  "watch": ["src"],
  "ext": "ts",
  "exec": "ts-node -r dotenv/config src/index.ts"
}
```

Explications

Étape N°14 : Expliquez chaque ligne comme précédemment

🎯 9. Ajouter les scripts dans `package.json`

Étape N°15 : Ajoutez ces scripts dans "scripts" :

```
"scripts": {
  "start": "node dist/index.js",
  "dev": "nodemon src/index.ts",
  "build": "tsc"
}
```

Explications

Étape N°16 : Expliquez chaque ligne comme précédemment

🚀 10. Lancer l'API

Étape N°17 : Démarrez l'API en mode développement :

```
npm run dev
```

Si tout fonctionne :

✅ Serveur démarré sur `http://localhost:4000`

🔧 11. Tester l'API

Utilise **Postman**, **cURL**, ou ton navigateur.

◆ **Tester GET /users**

```
curl -X GET http://localhost:4000/users
```

Réponse attendue :

```
{  
  "message": "Liste des utilisateurs"  
}
```

◆ Tester `POST /users`

```
curl -X POST http://localhost:4000/users -H "Content-Type: application/json" -d  
'{"name": "Alice", "email": "alice@example.com"}'
```

Tests

Étape N°18 : 🔍 1. Vérifier si l'API fonctionne

🔧 2. Tester avec `curl`

Étape N°19 : Testez avec curl :

◆ Tester `GET /users`

```
curl -X GET http://localhost:4000/users
```

◆ Tester `POST /users` (Corrigé)

```
curl -X POST http://localhost:4000/users -H "Content-Type: application/json" -d  
'{"name": "Alice", "email": "alice@example.com"}'
```

🚀 4. Tester avec `Postman` (Alternative)

Étape N°20 : Avec postman

1. Créer une nouvelle requête.
2. Choisir `POST`.
3. Entrer l'URL : `http://localhost:4000/users`
4. Aller dans l'onglet "Body" → Choisir "raw" → Sélectionner "JSON".
5. Entrer les données JSON :

```
{  
  "name": "Alice",  
  "email": "alice@example.com"  
}
```

✓ Que faire en cas d'erreur (port déjà utilisé) ?

Il faut vérifier les processus qui utilisent le port 4000

◆ Sur Windows

Ouvre une invite de commande (cmd ou Powerell) et tape :

```
netstat -ano | findstr :4000
```

Cela affichera un processus actif sur le port 4000, avec un identifiant **PID**.

Ensuite, tue ce processus avec :

```
taskkill /PID <NUMÉRO_PID> /F
```

(Remplace <NUMÉRO_PID> par le numéro trouvé dans la commande précédente.)

◆ Sur Mac/Linux

Exécute :

```
lsof -i :4000
```

Cela affiche un processus actif sur le port 4000.

Pour le fermer :

```
kill -9 <NUMÉRO_PID>
```

(Remplace <NUMÉRO_PID> par le numéro trouvé.)

Suite du TD / TP

✓ 1. Ajouter un stockage en mémoire (solution simple)

Étape N°21 : Modifions `user.controller.ts` pour utiliser un **tableau en mémoire** :

◆  `src/controllers/user.controller.ts` (Mise à jour)

Étape N°22 : Ajoutez une variable `users` pour stocker les utilisateurs :

```
import { Request, Response } from 'express';

const users: { name: string; email: string }[] = []; // Stockage en mémoire

export const getUsers = (req: Request, res: Response) => {
  res.json({ users });
};

export const addUser = (req: Request, res: Response) => {
  const { name, email } = req.body;

  if (!name || !email) {
    return res.status(400).json({ message: "Nom et email requis" });
  }
}
```

```

}

users.push({ name, email }); // Ajout en mémoire
console.log("👤 Utilisateur ajouté :", { name, email });

res.json({ message: `Utilisateur ${name} ajouté avec succès !`, email });
};

```

✅ 2. Tester le stockage

✅ 3. Stocker les utilisateurs dans une base de données (option avancée)

Si tu veux une solution **persistante**, on peut :

- Utiliser **MongoDB**
- Utiliser **MariaDB**
- Utiliser **SQLite**

Étape N°23 : Choisir une de ces possibilités et la mettre en œuvre

Étape N°24 : Écrivez les routes CRUD

Méthode	Route	Description
POST	/users	Ajouter un utilisateur
GET	/users	Récupérer tous les utilisateurs
GET	/users/:id	Récupérer un utilisateur spécifique
PUT	/users/:id	Mettre à jour un utilisateur
DELETE	/users/:id	Supprimer un utilisateur